

Firewalld Othmane Elmqiddem

Firewall

A firewall is a network security system that monitors and controls incoming and outgoing network traffic based on predefined rules. It operates at different layers of the network stack and relies on the kernel to enforce these rules. The kernel, being the core of the operating system, intercepts and evaluates data packets against the firewall's rules before they reach the application layer or the network interface, ensuring security.

Firewalld in RHEL 9

Firewalld is a dynamic firewall management tool in RHEL 9 that leverages the kernel's packet-filtering capabilities (via `nftables` or `iptables`). It provides an abstraction layer to define zones and apply rules dynamically, without requiring a system restart. The kernel enforces these rules, intercepting and managing packets at the network stack level based on the configurations set by Firewalld. This close interaction with the kernel ensures high performance and system-level control over network traffic.

Firewall services

What Are Services?

In the context of computing and operating systems, **services** refer to background processes or programs that perform specific functions to support other software, system operations, or user applications. These services typically run without direct user interaction and are managed by the operating system.

Firewall-cmd --list-services

we can also see what one of these services is :

Firewall-cmd --info-service ssh

We can also add services , let's for exemple add a web service :

Firewall-cmd --add-service http (we can also remove it by replacing add by remove)

If we want to do socket layer web traffic we will need to add the https service.

Attention setting up a firewall it can sometimes be dangerous specially when you're working on a remote machine , you can somehow block all system traffic if you do the wrong configuration

. Thats why all command we write and changes we made are persissted in memory only , so in case you broke your machine you can just reboot and everything will be fine .

In case we're sure of our configuration and we want it to be permanent we add at the end of each command an additional flag (--permanent)

Firewall Zones

Firewall zones are predefined or user-defined logical groupings in a firewall configuration that represent different levels of network trust. Each zone is associated with specific firewall rules, determining how network traffic is managed (allowed, denied, or restricted) based on the trust level of the source network or interface.

So with the firewall commands that we just started going through we were interacting with what is known as the default zone.

zones within firewalls can apply to different network cards , different ip ranges so you could have an internal network or a an external network.

So for exemple we can get active zones and that will tell us what zones are actually configured on the system a lot of times you'll see kind of a local subnet you'll see maybe an internal network within your data center and then you might see public internet.

So let's run this command :

```
Firewall-cmd --get-active-zones
```

We can also see the default zone in case we have multiple active zones (the default zone is the one that all changes will be applied to unless we specify in our command)

```
Firewall-cmd --get-dafault-zone
```

There's also this command that allows us to see all defiened zones :

```
Firewall-cmd --get-zones
```

We also can see the differnet info on a speciefic zone using this command :

```
Firewall-cmd --info-zone (name of the zone)
```

Target

- **Definition:** Default action applied to traffic not explicitly allowed or denied.
- **Options:**
 - **ACCEPT:** Allows traffic.
 - **DROP:** Silently drops traffic.
 - **REJECT:** Drops traffic and sends an error message.
- **Purpose:** Acts as a fallback for unhandled traffic.

- **Example:** Setting the `DROP` target ensures all unspecified traffic is silently discarded.

Rich Rules

- **Definition:** Advanced custom rules offering granular control over traffic.
- **Purpose:** Allows complex configurations beyond basic services and ports.
- **Example:** A rich rule to allow traffic from a specific IP range only during certain hours.

Protocols

- **Definition:** Network protocols (e.g., TCP, UDP) allowed or blocked in the zone.
- **Purpose:** Ensures traffic adheres to the desired communication standards.
- **Example:** Allowing UDP traffic for DNS requests while blocking others.

Multi-ports services:

one of the nice things about the idea of a service when it comes to firewall d on red hat enterprise linux so you know in the old days we maintained our firewall settings and we would have to manually put-in rules for every single port that we wanted to allow or refuse or whatever the case may be , but with firewall command and firewall d we can group those things together into these services

So for exemple let's run this command :

```
Firewall-cmd --info-service samba
```

The **Samba service** is an open-source implementation of the SMB/CIFS protocol, enabling file and printer sharing between Linux/Unix systems and Windows devices. So it is acting as a file server or domain controller.

so if i want to allow samba traffic to get through my firewall i just add the service and that adds rules so those ports , that by running the following command :

```
Firwall-cmd --add-service samba
```

Allowing ports

So that's makes us wonder what if i've written some homebrew applications something that is specific only to my uh environment is there a way that i could perhaps add-in or open up

ports in my firewall that aren't part of an official service .

The answer is yes , so for exemple let's say i want to change the port for ssh which is usually defined as port 22 , let's make it port 2022. well we can do that by running the following command :

```
Firewall-cmd --add-port=2020/tcp
```

Scanning ports using nmap

so if i wanted to troubleshoot from one system to another there's a number of different utilities and we want to highlight one of them specifically to ports um so if we've got a destination server that we've changed the ssh port from from 22 to 2022 like we have here if i'm sitting on my local machine and i'm trying to get to that remote machine i can actually use a package called nmap to basically pull a destination host and see what ports are available so if i were trying to connect from this box to another box i could actually do an nmap and then the ip address and that command will actually return back a list of ports ,it'll essentially do a quick port scan that will tell me what ports are listening

and before we do this i feel compelled to point out that nmap works by sending a storm of traffic to your destination right so i'm doing an nmap against my public ip and really what it's going to do is just trying to connect to every single port on that ip which depending on the area in which you work may cause interesting side effects so if you work at a place where your network team does a lot of traffic monitoring and traffic shaping they will likely notice that you have run this and it will set off all kinds of flags in their network operations center because there's this like huge volume of traffic that goes to every port on a host which is a hallmark for something like someone trying to do a scan or an attack on a host so depending on where you work or depending on what machines you're using first you want to make sure that it is permissible for you to use a tool like nmap

So first we run the command :

```
Ip a
```

Then the command :

```
nmap (ip)
```

So there is a file that contains a list of ports and what they are

The file is :

```
less /etc/services
```